Lecture 18: Security

Some slides borrowed from:
Tyler Bletsch (NC State), Berkeley CS61C, Branden Ghena (Northwestern)

Today’s Goals

- Introduce OS security considerations.
- Describe memory-based attacks and defenses.
- Explore speculative execution attacks and ramifications.

Why is computer security so important?

- Most public security happens at least in some portion on the honor system
  - Pretty easy to break a window
  - Keyed locks are easy to pick
  - Master keys can be determined and manufactured (Matt Blaze attack)
  - Laws apply after you've done it

Early computers didn’t have any security either

- Simple machines for doing computation do not have private files or contention

- Timeslicing machines meant there were multiple users, but all were employees of the same company
  - Permissions needed to be as secure as a file in a locked drawer on a desk

“The act of breaking into a computer system has to have the same social stigma as breaking into a neighbor's house. It should not matter that the neighbor's door is unlocked.”
- Ken Thompson, Turing Award Lecture, 1984

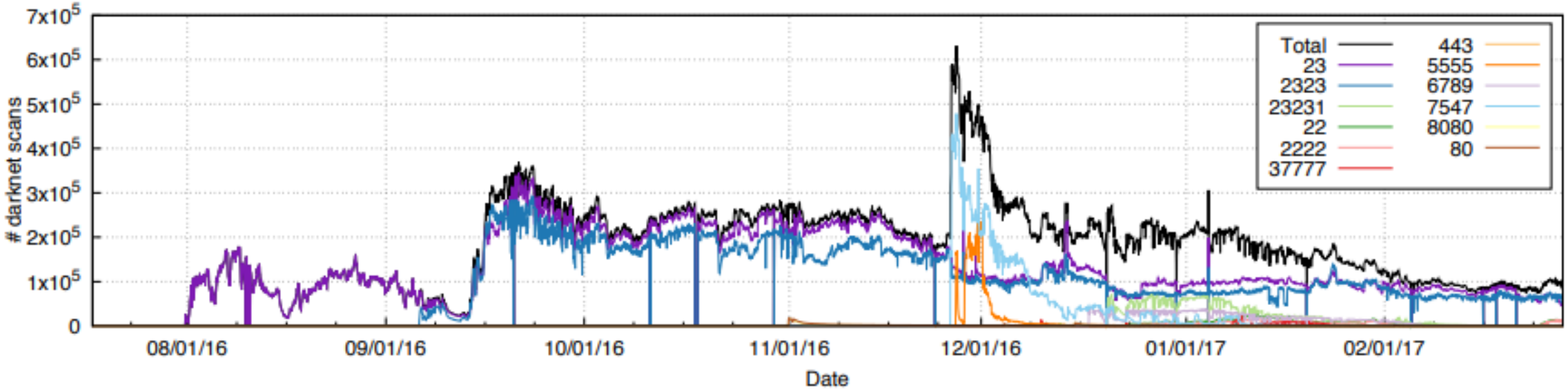
Connectivity of computers makes security a top concern

- Importantly, physical item security is dependent on the fact that one person can only steal one thing at a time
  - And it's usually obvious when theft occurs

- The internet changed all of this for computers
  - Usually not people breaking into computers manually, one at a time
  - Instead it is computers breaking into computers by means of scripting
  - And you can access a computer from anywhere on Earth

- Breaking into or controlling one car is a crime
  - Controlling 100,000 cars remotely is a problem for the manufacturer

Mirai botnet (2016)

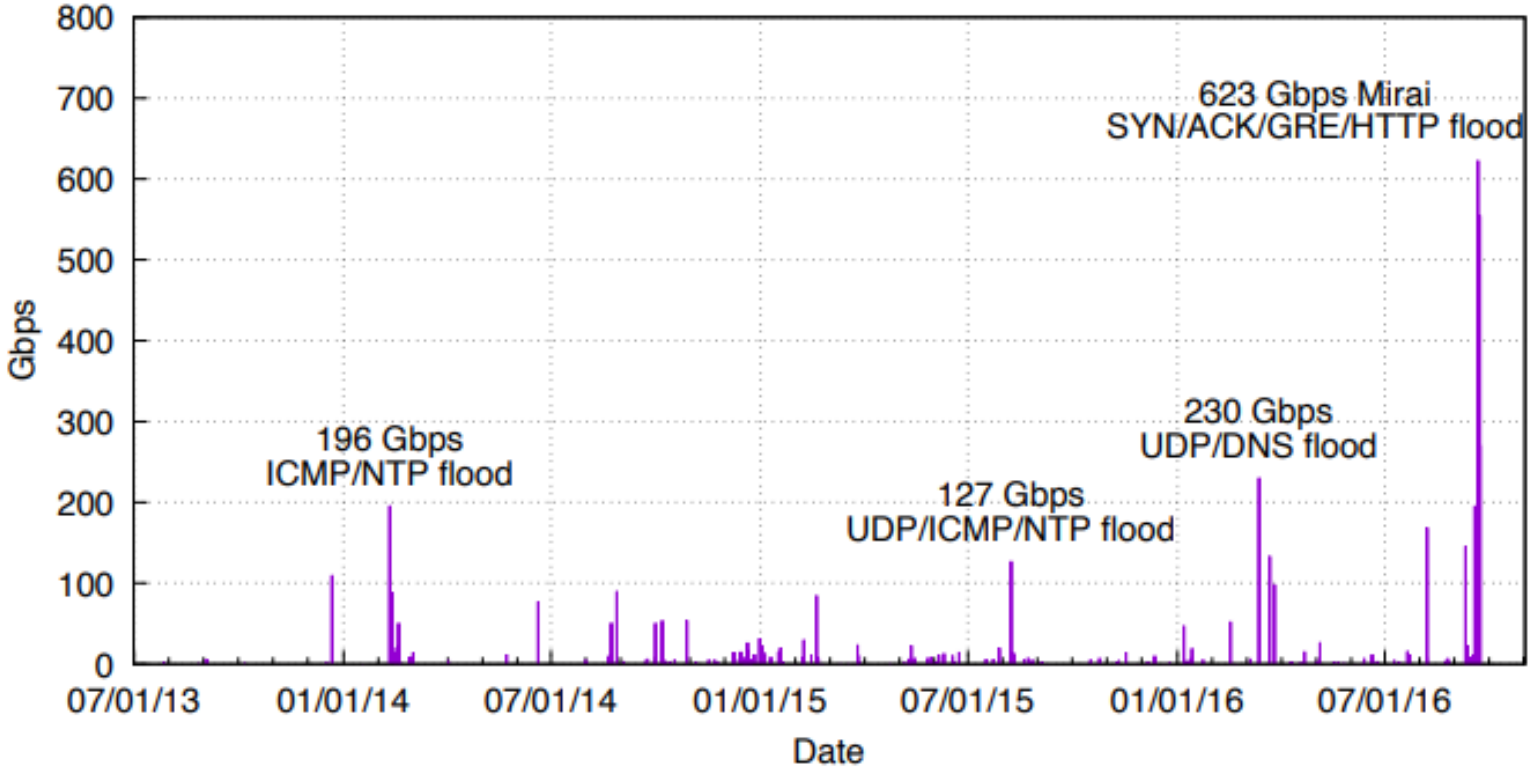


- Takes control of up to 600,000 insecure connected devices
  - IP-attached cameras, DVRs, routers, printers

Botnets can be directed towards denial-of-service attacks

- Mirai is used for DDOS attacks on various websites
  - Krebs on Security blog gets 623 Gbps of traffic during one attack

DDOS attacks targeting Krebs on Security



- **Design for security**

- Memory attacks and defenses
  - Buffer overflow and No-Execute bit
  - Return-Oriented Programming and Address Space Layout Randomization

- Speculative execution attacks
  - Meltdown
  - Spectre

Trusted Computing Base (TCB)

- Trusted Computing Base is everything the OS relies on to enforce security
  - If everything outside of the TCB is “evil”, the TCB can still be trusted
  - Important to be a clear, minimum set of components

- TCB includes
  - Scheduler, Memory Management, Parts of file system, Parts of device drivers

- Anything else must be assumed malicious
  - Processes memory accesses, System call arguments, Received packets

Modern code bases are enormous

Program/Use Case	Millions of Lines of Code
Unix v1.0	0.01
Average iPhone app	0.04
Space Shuttle	0.4
Windows 3.1	2.5
Mars Curiosity Rover	5
Firefox (2015)	9.7
F-35 Fighter jet	24
Microsoft Office 2001	25
Windows 7	40
Facebook (2015)	62
Debian 5.0 codebase	68

https://www.informationisbeautiful.net/visualizations/million-lines-of-code/

- For many projects, no one person has read and understood all of it
- TCB needs to be agreed upon by everyone working on the project
  - And needs to enforced by everyone in the project

Writing auditable code

- Code style and semantics really do matter!!

- If you want code to be secure, it needs to be read AND understood by many people

- Bad code style/semantics builds up cognitive load of the reader making them less likely to notice when something is wrong
  - 0 versus NULL
  - &buf[0] versus &(buf[0])
  - int x, y, z; versus int x; int y; int z;

Apple “goto fail” SSL bug

This code is actually decently commented, just not in this particular section.

```
...
 if ((err = SSLFreeBuffer(&hashCtx)) != 0)
        goto fail;

    if ((err = ReadyHash(&SSLHashSHA1, &hashCtx)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
        goto fail;
        goto fail;
    if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
        goto fail;
...
```

Apple “goto fail” SSL bug

Spacing intentional. This code mixes tabs and spaces and has random extra line breaks.

It is actually decently commented, just not in this particular section.

```
...
 if ((err = SSLFreeBuffer(&hashCtx)) != 0)
        goto fail;

    if ((err = ReadyHash(&SSLHashSHA1, &hashCtx)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
        goto fail;
        goto fail;
    if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
        goto fail;
...
```

Apple "goto fail" SSL bug

Spacing intentional. This code mixes tabs and spaces and has random extra line breaks.

It is actually decently commented, just not in this particular section.

```
...
 if ((err = SSLFreeBuffer(&hashCtx)) != 0)
        goto fail;

    if ((err = ReadyHash(&SSLHashSHA1, &hashCtx)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
        goto fail;
        goto fail;
    if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
        goto fail;
...
```

Outside of IF statement!! Always runs.

Security properties OS should enforce

- Confidentiality
  - Private information should remain private
  - Example: processes can’t read memory in another process

- Integrity
  - Mechanisms should not be modified without permission
  - Example: OS data structures (e.g., file table entries) can’t be modified by processes

- Availability
  - Resources on the computer should be able to be fairly accessed
  - Example: network access is shared among processes

OS security concerns: Enforcing Access

- Processor access
  - Integrity: User versus kernel mode
  - Availability: Timeslicing

- Memory access
  - Confidentiality and Integrity: Virtual memory (and permissions)
  - Availability: Swapping pages via issuing pagefaults

- File access
  - Confidentiality: Permissions (user and group)
  - Integrity: only accessible through system calls

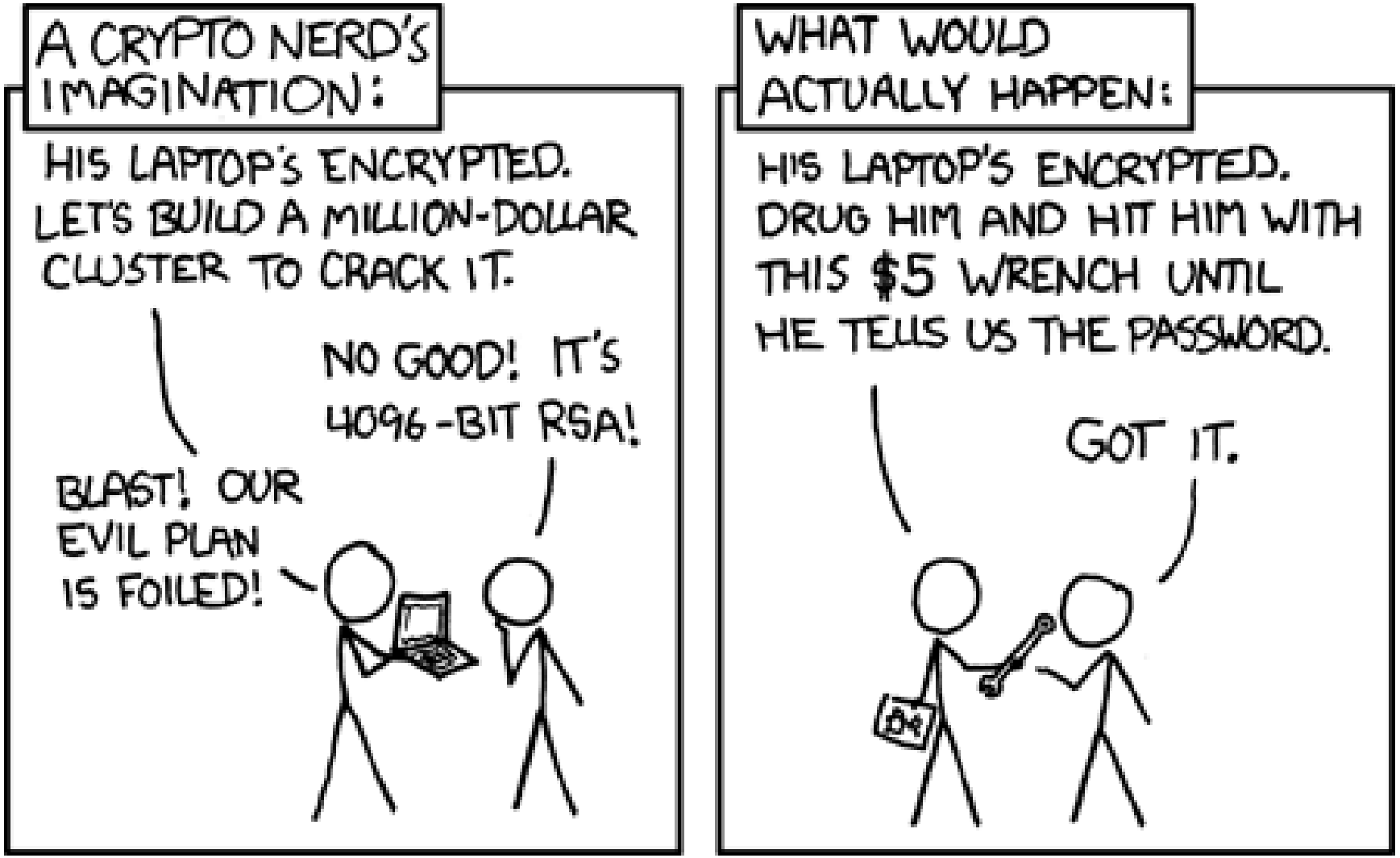
Security is an arms race

- There is no single fix for system security
  - New attacks are constantly being discovered
  - New solutions are constantly being applied

1. Find a vulnerability and how it can be exploited
2. Fix vulnerability
3. Go back to 1

- But if the OS is designed with security in mind, it's hopefully harder to find vulnerabilities in the first place

xkcd



https://xkcd.com/538/

- Design for security

- **Memory attacks and defenses**
  - **Buffer overflow and No-Execute bit**
  - **Return-Oriented Programming and Address Space Layout Randomization**

- Speculative execution attacks
  - Meltdown
  - Spectre

What’s wrong with this code?

```
#include <stdlib.h>
#include <stdio.h>

int main() {
     char name[1024];
     printf("What is your name? ");
     scanf("%s", name);
     printf("%s is cool.\n", name);

     return 0;
}
```

Buffer overflow potential with “nice” input

```
tkbletsc@davros:~/jop/examples/code-injection $ ./cool
What is your name? Tyler
Tyler is cool.
tkbletsc@davros:~/jop/examples/code-injection $
```

Buffer overflow potential with “evil” input

```
tkbletsc@davros:~/jop/examples/code-injection $ ./cool < attack
What is your name? ⍰⍰⍰⍰⍰⍰⍰Ph... hpeed⍰⍰⍰⍰⍰⍰P⍰⍰⍰⍰⍰⍰⍰⍰Phhinghomet⍰⍰⍰⍰⍰⍰P⍰⍰⍰⍰⍰⍰⍰⍰P⍰⍰⍰⍰⍰⍰⍰⍰P⍰⍰⍰⍰
⍰⍰⍰P⍰⍰⍰⍰⍰⍰P⍰⍰⍰⍰⍰⍰⍰⍰P⍰⍰⍰⍰⍰⍰ ⍰⍰⍰P⍰ ⍰⍰⍰⍰P⍰⍰⍰⍰⍰⍰P⍰⍰⍰⍰⍰⍰P⍰⍰⍰⍰⍰⍰Phren'⍰⍰⍰⍰⍰⍰Phlear⍰⍰⍰⍰⍰⍰Ph;1mYh [31,
ü⍰⍰⍰⍰ ⍰⍰⍰⍰⍰ ⍰⍰⍰⍰⍰ ⍰⍰⍰⍰⍰⍰⍰⍰Phtar.h2.7.hhon-h/Pyth/2.7hthonhp/pyhg/fthn.orhythohww.ph://whhttp1⍰P
⍰wgetP⍰⍰1⍰S-⍰⍰⍰⍰P⍰⍰⍰⍰P⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰ Ph/wgeh/binh/usr⍰⍰⍰⍰⍰⍰ ⍰⍰⍰⍰⍰ xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰⍰ is cool.
You clearly aren't cut out for C.  How about I start you off on something more your speed...

--2010-09-22 11:40:00--  http://www.python.org/ftp/python/2.7/Python-2.7.tar.bz2
Resolving www.python.org... 82.94.164.162, 2001:888:2000:d::a2
Connecting to www.python.org|82.94.164.162|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 11735195 (11M) [application/x-bzip2]
Saving to: `Python-2.7.tar.bz2'

100%[=====================================================>] 11,735,195  3.52M/s   in 3.8s

2010-09-22 11:40:05 (2.97 MB/s) - `Python-2.7.tar.bz2' saved [11735195/11735195]
```

Buffer Overflow

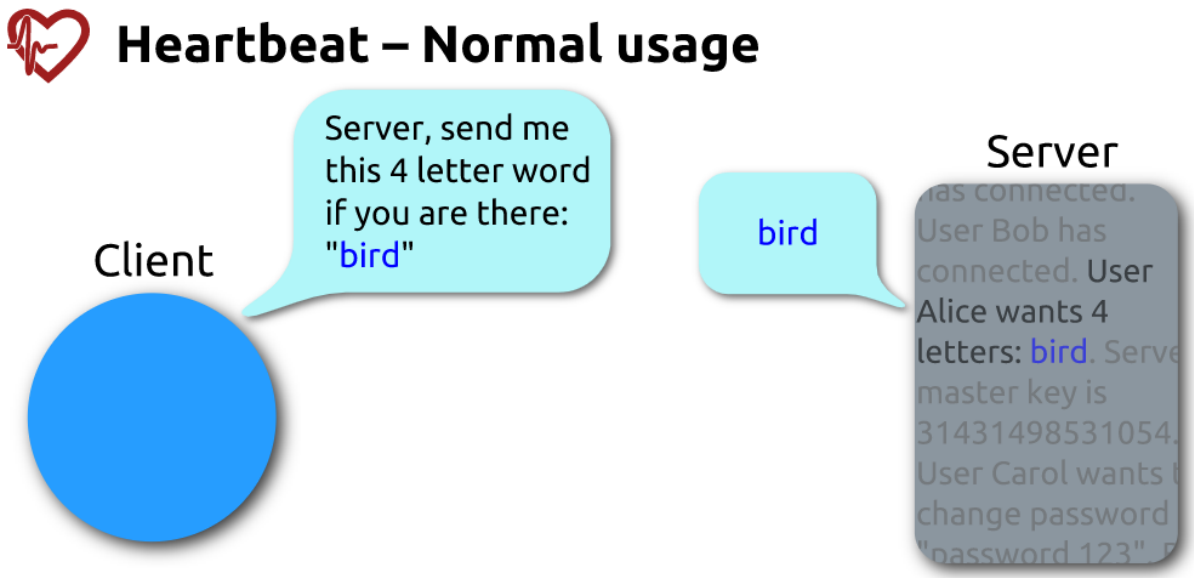
- Arrays (buffers) in C are not bounds checked
  - Can keep writing past the end of the array
  - Overwrites either data section or stack section

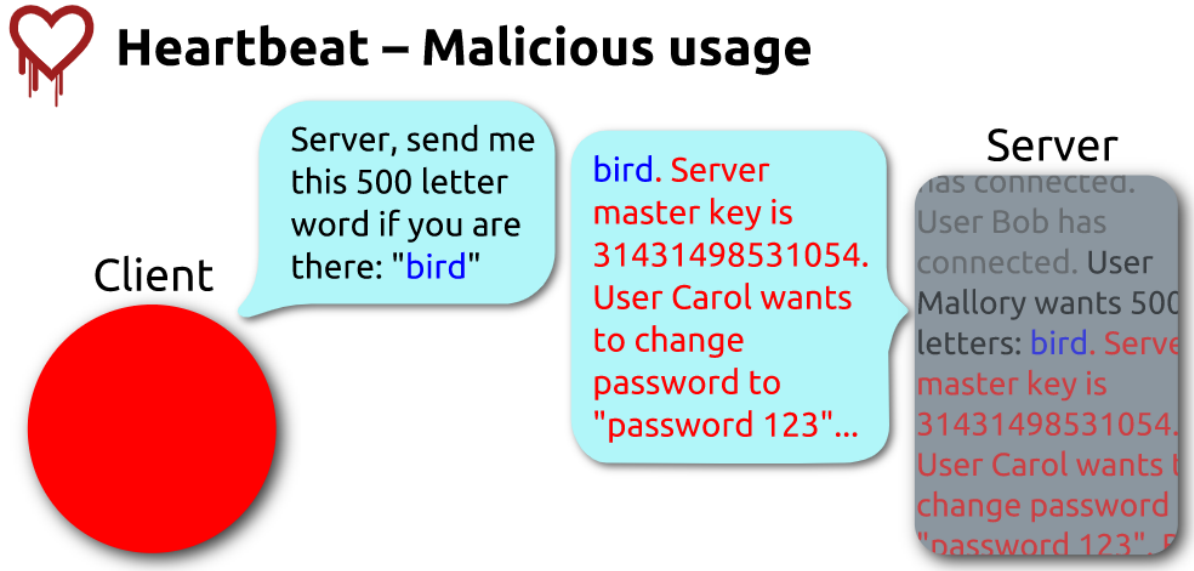
- Still an incredibly common problem in C

- Key problem
  - Trusting input from an untrustworthy source
  - Users are not part of the trusted computing base
    - Certainly not arbitrary inputs they can make

Heartbleed attack

- Vulnerability in OpenSSL
  - 2014
- Started the trend of vulnerabilities with cool names and logos





Extra: Unsafe C library functions (and replacements)

`gets(char *str)`	read line from standard input into str
`sprintf(char *str, char *format, ...)`	create str according to supplied format and variables
`strcat(char *dest, char *src)`	append contents of string src to string dest
`strcpy(char *dest, char *src)`	copy contents of string src to string dest
`vsprintf(char *str, char *fmt, va_list ap)`	create str according to supplied format and variables

Better choices:

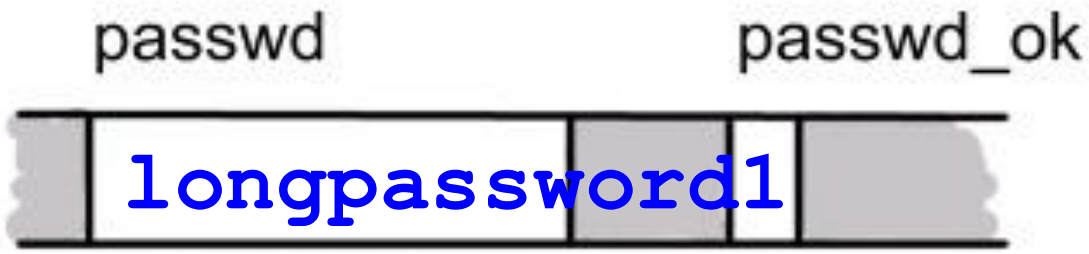
```
char *fgets(char *s, int size, FILE *stream)
snprintf(char *str, size_t size, const char *format, ...);
strncat(char *dest, const char *src, size_t n)
strncpy(char *dest, const char *src, size_t n)
vsnprintf(char *str, size_t size, const char *format, va_list ap)
```

Buffer overflows can overwrite important variables

- Long input string can overwrite variables on the stack
  - Such as the password check

```
int main(int argc, char *argv[]) {
    char passwd_ok = 0;
    char passwd[8];
    strcpy(passwd, argv[1]);
    if (strcmp(passwd, "niklas")==0)
        passwd_ok = 1;
    if (passwd_ok) { ... }
}
```

Might also modify whole functions!



Morris Worm

- November 02, 1988
  - Roughly 88,000 computers on internet at the time
- Worm
  - Invading program that installs itself on additional computers
- Infected several thousand computers, taking down internet for several days

How the worm entered computers

1. Debug vulnerability in *sendmail* - email sending service
   - Connect, enter debug mode, send arbitrary code to execute

2. Buffer overflow in *finger* – lists users on server
   - Send request with more than 512 bytes of arguments
   - Execute /bin/sh

3. Guess passwords
   - Get list of users for the machine worm is already running in
   - Guess username, reverse username, 400 “popular” words, entire dictionary
     - Reuse, use it on other computers on the network

Effects of Morris Worm

- Morris Worm created too many copies of itself
  - Checked if there was already a worm on the computer before running
  - 1/7 of the executables ran anyways (reinfection)

- Computers ended up with many processes running
  - How are too many processes harmful?

Effects of Morris Worm

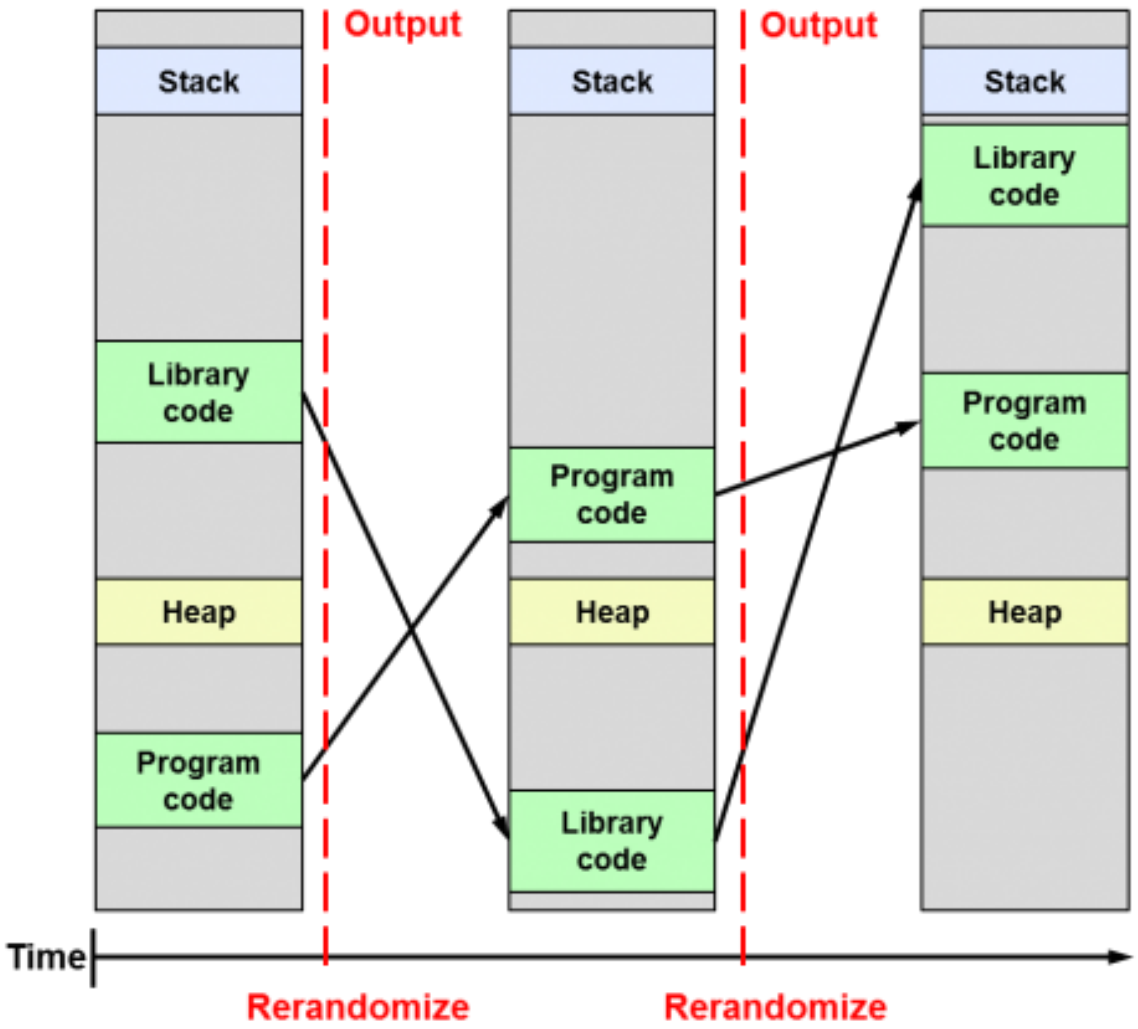
- Morris Worm created too many copies of itself
  - Checked if there was already a worm on the computer before running
  - 1/7 of the executables ran anyways (too high a default)

- Computers ended up with many processes running
  - Long response time due to so many processes
  - Thrashing due to too much memory pressure
  - Slowed computers to a halt

- Outcomes:
  - Invaded ~6000 computers in hours (10% of the Internet at the time)
  - CERT was created to manage software security
  - First Computer Fraud and Abuse Act (CFAA) prosecution

Address-space layout randomization (ASLR)

- Randomize memory region locations in virtual memory
  - Already spread throughout physical memory
- Move locations of libraries and code relative to each other
  - Arbitrary address for attacker to send code to gets harder to predict!
- Implemented 2005-2007
  - Linux, MacOS, and Windows
  - 2011 for Android and iOS



Running a process again re-randomizes its layout

CVE

- The Common Vulnerabilities and Exposures (CVE) system documents publicly released software vulnerabilities.
- How long has it been since the last CVE due to a buffer overflow?

CVE-2026-40026

The Sleuth Kit through 4.14.0 contains an out-of-bounds read vulnerability in the ISO9660 filesystem parser where the parse_susp() function trusts len_id, len_des, and len_src fields from the disk image to memcpy data into a stack buffer without verifying that the source data falls within the parsed SUSP block. An attacker can craft a malicious ISO image that causes reads past the end of the SUSP data buffer, and a zero-length SUSP entry can trigger an infinite parsing loop.

Source: VulnCheck

Max CVSS	4.8
EPSS Score	N/A
Published	2026-04-08
Updated	2026-04-08

CVE-2026-40025

The Sleuth Kit through 4.14.0 contains an out-of-bounds read vulnerability in the APFS filesystem keybag parser where the wrapped_key_parser class follows attacker-controlled length fields without bounds checking, causing heap reads past the allocated buffer. An attacker can craft a malicious APFS disk image that triggers information disclosure or crashes when processed by any Sleuth Kit tool that parses APFS volumes.

Source: VulnCheck

Max CVSS	4.8
EPSS Score	N/A
Published	2026-04-08
Updated	2026-04-08

https://www.cvedetails.com/vulnerability-list.php

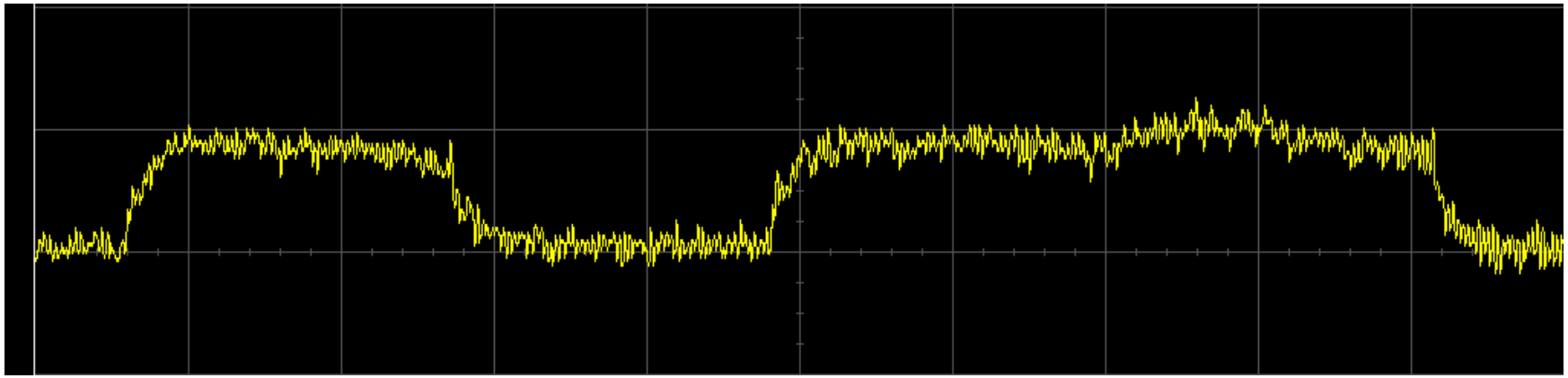
- Design for security

- Memory attacks and defenses
  - Buffer overflow and No-Execute bit
  - Return-Oriented Programming and Address Space Layout Randomization

- **Speculative execution attacks**
  - **Meltdown**
  - **Spectre**

Background: Side channel attacks

- Many physical systems have properties that may leak information about internal state
  - Determine RSA key bits based on power use during a decrypt operation
  - Determine length of password by how long it takes to check it



Timing attacks are one side channel

- Timing attacks can be overcome with constant-time algorithms which always take as long as the worst-case execution time
  - But this means reducing performance

- Caches are essentially one big timing attack
  - Speeds up access to data if it is present in the cache
    - This was the goal!!
  - An attack can know which data was accessed recently

Background: Speculative Execution

Modern processors want to always be doing something

- What if we're going to branch based on a memory load?
- What if we just guess what the result will be and start executing early!

So they are often "speculatively executing" instructions

- Perform the operation and throw out the result if we shouldn't actually do it
- For example, branch prediction

Optimization: Kernel Mapped in Virtual Memory

Page tables map virtual memory to physical memory for a process

But actually, we often leave the OS memory in the page table too…

- Faster to switch back to the OS

Meltdown

Security vulnerability in all modern processors that allows arbitrary reads from memory

Disclosed in January 2018 by:

- Jann Horn (Google Project Zero),
- Werner Haas, Thomas Prescher (Cyberus Technology),
- Daniel Gruss, Moritz Lipp, Stefan Mangard, Michael Schwarz (Graz University of Technology)

Details:

- https://hackernoon.com/a-simplified-explanation-of-the-meltdown-cpu-vulnerability-ad316cd0f0de
- https://meltdownattack.com/meltdown.pdf

Step 1: Read from a kernel address

```
mov $KERNEL_ADDRESS_OF_SECRET, %r12
mov (%r12), %eax
```

%eax now holds a byte of memory that we shouldn't able to access

- This will be an invalid page fault!
- BUT, once the instruction actually hits the end of the pipeline!

- For now, it loads that value into %r12 right away and continues executing speculatively

Step 2: Read based on secret

```
mov $KERNEL_ADDRESS_OF_SECRET, %r12
mov (%r12), %eax
mov MY_ARRAY(%eax), %edx
```

%edx is a valid read from our own memory

- This is never going to finish either because the process will have an exception from the prior instruction, but it will start executing...

Step 3: Handle the Exception

```
 mov $KERNEL_ADDRESS_OF_SECRET, %r12
```
~~`mov (%r12), %eax`~~
~~`mov MY_ARRAY(%eax), %edx`~~

The processor realizes you tried to read from memory you didn't have access to and generates an exception

- You can catch these and recover
- The invalid instruction and ones after it are rolled back as if they never happened

Everything's still safe right?

The processor never saved any results from the invalid accesses to memory in registers

- So there's no problem, right?

We forgot about the cache

The load affected the cache!!!

```
mov $KERNEL_ADDRESS_OF_SECRET, %r12
mov (%r12), %eax
mov MY_ARRAY(%eax), %edx
```

The value at address **`MY_ARRAY+%eax`** was saved in our cache

Step 4: Time loads from memory

```
for (int i=0; i<255; i++){
    start_time = time();
    int temp = MY_ARRAY[i*CACHE_BLOCKSIZE];
    stop_time = time();

    if ((stop_time-start_time)==short_time){
        secret = i;
    }
}
```

The cache speeds up the access to the one memory address that was cached due to speculative execution

Step 5: Repeat and Profit

- Now we know the value of a single byte

- But we can repeat this process over and over to read arbitrary memory
  - Read from memory at ~500 kbps

- Incredible part is how relatively simple this attack is
  - Does require systems knowledge of multiple domains
  - Computer architecture, OS, and security

How do we fix this?

1. Stop speculatively executing
   - Already in the hardware
   - Would slow all computers down a lot
2. Stop caching speculative loads
   - Already in the hardware
   - Would slow all computers down a lot
3. Kernel Page Table Isolation ✓

Ramifications of speculative execution attacks

- Particularly big deals in the era of cloud computing
  - Anyone can run a program on an AWS server
  - And now can maybe read data from the other running programs…

- Speculative execution attacks are a new era for computer security
  - Hardware is still being actively developed to address attacks
    - Websites can be fixed in hours, Programs in days, OSes in weeks, and Hardware takes years
  - Attacks are still being developed
  - OS continues to have to adapt to both sides

- Design for security

- Memory attacks and defenses
  - Buffer overflow and No-Execute bit
  - Return-Oriented Programming and Address Space Layout Randomization

- Speculative execution attacks
  - Meltdown
  - Spectre